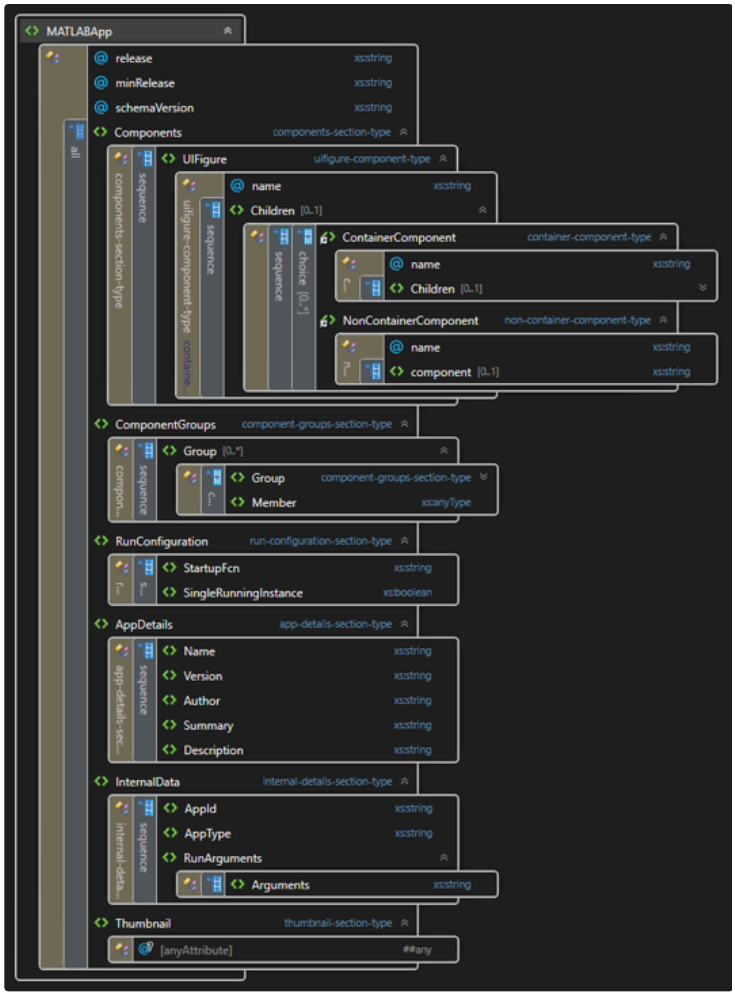
Plain Text App XML Schema Overview (26b)

[Plain Text App XML Schema Overview (26b) - MATLAB App Designer - Confluence]

The Plain Text app XML describes a MATLAB App in a way that closely mirrors what users see and work with in **App Designer**. Rather than encoding executable code, it captures the **structure, components, layout, and configuration** of an app in a declarative form.



Plain Text App XML structure and vocabulary overview (image generated from XSD)

Note: The terms “required” and “optional” used in this document are an indication of whether an app will run and load into App Designer.

`<MATLABApp>` (required)

The root of the XML document. Everything MATLAB needs to run the app or App Designer needs to load the app lives beneath this node.

The direct children of `<MATLABApp>` correspond to categories of data needed to fully represent a MATLAB App. These elements are ordered such that information important to the user appears closer the top of the file:

- `<Components>` A hierarchy of app **components** and their configuration (*required*)
- `<ComponentGroups>` **Component groups** defined by the user for design-time convenience (*optional*)
- `<RunConfiguration>` **Run & startup** behavior (*optional*)
- `<AppDetails>` Descriptive **information for sharing** the app with others (*optional*)
- `<InternalData>` **Data needed to load** the app as expected in App Designer (*optional*)
- `<Thumbnail>` An **image** for easier visual identification (*optional*)

Attributes

- `schemaVersion` (optional)
  - XML vocabulary and structure used. Considered an XML de facto standard to include schema version information in the XML root.
  - Semantic version format (MAJOR.MINOR.PATCH)
  - *default:* `'1.0.0'`
- `release` (optional)
  - MATLAB release and update used to save the app (optional; default 'R2026b.0')
  - `release.update(0-N)` format, e.g., `'R2031a.2`
  - *default:* `'R2026b.0'`
- `minRelease` (optional) — minimum MATLAB release required to **load** the app in App Designer
  - *default:* `'R2026b'`

The **schema version** is a format-level contract and informs how the XML is *parsed and validated*.

The **release information** is a MATLAB release-level contract and informs how the resulting data are *interpreted and processed* for run and/or load.

`<Components>` (required)

The `<Components>` element of a MATLAB app contains **exactly one** `<UIFigure>`. This is the root of the app's component hierarchy.

Every component descending from `<UIFigure>` is represented as an **XML element** whose name matches the corresponding component's MATLAB class name. This reflects the mental

model users already have from App Designer and the underlying app-building MATLAB API.

For example:

- `<Button>`
- `<Panel>`
- `<UIAxes>`
- `<NumericEditField>`

User-authored Components (UAC) are similarly serialized in the XML using their class name. To help prevent component name collisions, if a UAC is namespaced (i.e., in a package), its fully qualified package name will be used.
For example:

- `<DynamoWidgets.FileSelector>`

Component Attributes

- `name` (required) — The component's CodeName
  - Seen in Component Browser, app's MATLAB code
  - Used as reference in the XML (e.g., `ContextMenu` assignment, component label)
- `label` (optional) — Label name associated with component (see [Labeled Components]()
)

Component Properties (Child Elements)

Each component's property configuration is serialized as **child elements** of the component. The property element names match the component's **property names seen in App Designer** (e.g., in the Property Inspector).

- This includes "nested properties" exposed in the Inspector, e.g., `Title.String`
- **Exception**: For `GridLayout` children the properties `<Layout.Row>` and `<Layout.Column>` are serialized, which are not directly exposed in App Designer, other than in generated MLAPP code. (Note in this case the `<Position>` property is *not* serialized.)

Component property elements are serialized in alphabetical order

- **Exception:** `<Children>` is always serialized **last**, regardless of alphabetical order.

Property values are written using MATLAB-style conventions (for example, numeric vectors, quoted character literals, and cell arrays), also reflecting what is seen in the Property Inspector.

See [Document-Global Characteristics]() for escaped characters in element and attribute values.

Only non-default values are serialized.

- Properties that match component defaults value are omitted. Note this is consistent with MLAPP code generation.

Example XML for a Switch and Button:

```
<Switch name='powerSwitch'>
    <FontAngle>'italic'</FontAngle>
    <FontColor>[0.1059 0.0902 0.2745]</FontColor>
    <FontName>'Lucida Fax'</FontName>
    <FontWeight>'bold'</FontWeight>
    <Items>{'Default data', 'Joe's data'}</Items>
    <Orientation>'vertical'</Orientation>
    <Position>[226 131 20 45]</Position>
    <Value>'Default data'</Value>
    <ValueChangedFcn>powerSwitchValueChanged</ValueChangedFcn>
</Switch>
<Button>
    <Position>[10 10 100 22]</Position>
    <Text>'Plot &amp; Analyze'</Text>
</Button>

```

`Children` **Property**

Container components **may contain child components**.

Example container components include:

- `Panel`
- `GridLayout`
- `Tab`
- `TabGroup`
- `Tree`
- `Menu`
- `Toolbar`
- `ButtonGroup`
- `ContextMenu`
- `TreeNode`

Container components can define children explicitly using a `<Children>` element.

- Children are listed as nested component elements.
- The child order reflects **creation order** (i.e., inverse of stacking order)
- The allowed children depend on the container:
  - `UIFigure` → Any component, `Toolbar`, `Menu`
  - `Toolbar` → `PushTool`, `ToggleTool`

- `Menu` → `Menu`
- `ContextMenu` → `Menu`
- `TabGroup` → `Tab`
- `Tree` → `TreeNode`
- Generic containers (e.g., `Panel`, `GridLayout`) → any supported component

Labeled Components

Labeled Components are an App Designer construct. They consist of:

- The main component (e.g., `EditField`)
- An associated `Label` component

In **App Designer**:

- These are treated in the Canvas as a **group**
- The Label is hidden in the Component Browser by default
- Users often conceptualize the labeled component as a single entity

In the **XML**:

- The label is a normal `Label` component
- The association is represented by a `label` attribute on the main component
- Only **user-created design-time groups** are serialized under `<ComponentGroups>`

```
<Label name='PhoneEditFieldLabel'>
    <HorizontalAlignment>'right'</HorizontalAlignment>
    <Position>[81 179 55 22]</Position>
    <Text>'Enter your phone number:'</Text>
</Label>
<EditField name='PhoneEditField' label='PhoneEditFieldLabel'>
    <Position>[151 179 100 22]</Position>
</EditField>

```

`<ComponentGroups>` (optional)

Contains user-created App Designer Canvas groups. Only groups explicitly created by the user are serialized.

Must contain **one or more** `<Group>` **elements**

- `<Group>` may one or more elements:
  - `<Member>`
    - Value: Component `name` (attribute)
  - `<Group>`

```
<ComponentGroups>
  <Group>
    <Member>PlotButton</Member>
    <Member>MainAxes</Member>
    <Group>
      <Member>SendDataButton</Member>
      <Member>NumericEditField</Member>
    </Group>
  </Group>
</ComponentGroups>

```

`<RunConfigurations>` (optional)

Defines app **startup and execution behavior**.

May contain elements:

- `<StartupFcn>`
  - MATLAB callback executed when the app starts
- `<SingleRunningInstance>`
  - `true` → app behaves as a singleton (only one instance may run at a time)

```
<RunConfigurations>
  <SingleRunningInstance>true</SingleRunningInstance>
  <StartupFcn>startupFcn</StartupFcn>
</RunConfigurations>

```

`<AppDetails>` (optional)

User-configured information intended for **app sharing workflows**. These values correspond to fields in the **App Details** dialog in App Designer.

May contain elements

- `Name`
- `Summary`
- `Description`
- `Author`
- `Version`

```
<AppDetails>
  <Author>Jane Doe</Author>
  <Description>'Analyzes sensor data</Description>
  <Name>Sensor Analyzer</Name>
  <Summary>Analyze and visualize sensor signals</Summary>
  <Version>5.2.0</Version>
</AppDetails>

```

`<InternalData>` (optional)

Contains **design-time metadata managed by App Designer**, required to load and edit the app correctly.

May contain elements

- `<AppId>`
  - Stable UUID identifying the app
- `<AppType>`
  - App classification (`Standard` , `Responsive`)
  - `Responsive` → App Designer load will perform additional validation and design-time configuration of the `UIFigure` → `GridLayout` → `Panel` hierarchy. App run-time behavior is unaffected.
- `<RequiredProducts>`
  - Licenses/toolboxes required to load or run the app
  - Must contain one or more `<Product>` elements
- `<RunArguments>`
  - Arguments used when running the app
  - Must contain one or more `<Arguments>` elements

```
<InternalData>
  <AppId>'b9130ba9-013c-4e0f-9a70-469602dba01a'</AppId>
  <AppType>'Standard'</AppType>
  <RequiredProducts>
    <Product>Aerospace_Toolbox</Product>
    <Product>audio_system_toolbox</Product>
  </RequiredProducts>
  <RunArguments>
    <Arguments>{1, 'launch'}</Arguments>
  </RunArguments>
</InternalData>

```

Document-Global Characteristics

Character Escaping

Element values

`<` (`%lt;`)

`>` (`>`)

`&` (`$amp;`)

Attribute values

`'` (`%apos;`)

`<` (`%lt;`)
`>` (`>`)
`&` (`$amp;`)
(note: **single quotes** are used for attribute values, **double quotes do not need to be escaped**)

Indentation

The user's MATLAB setting for "Tabs and Indents" is applied to the XML and is consistent with the MATLAB file and App Designer's Code View.

Previously Reviewed & Supporting Documents

MAPP Appendix Design: Elements and Attributes - MATLAB App Designer - Confluence

Plain Text App: Appendix High level structure - MATLAB App Designer - Confluence

Alternative Approaches - MATLAB App Designer - Confluence

App Config XML - MATLAB App Designer - Confluence

MAPP File Content - Scratchpad - MATLAB App Designer - Confluence